

# Backend + AI Agent Developer Take-Home Assignment

## The Brief

Build a **Persistent Sales Assistant Agent** — a hosted conversational API where the agent remembers context across sessions, uses real tools to answer questions about a product catalog, and produces a self-evaluation score on every response.

Most "AI chat" demos are stateless. They forget you the moment you close the tab. The real differentiator in production agents in 2026 is memory + eval. That's what this tests.

## Scenario

You're building the backend for an early-stage B2B SaaS. The sales team needs an AI assistant that:

- Knows your product (a mock SaaS pricing and features catalog you provide)
- Remembers each user's questions and interests across multiple sessions
- Scores its own responses so bad answers can be caught before a human sees them
- Exposes clean API endpoints the frontend team can wire up in a day

## Endpoints to Build

POST /chat/{user\_id} → send a message, get a response + self-eval score  
GET /chat/{user\_id}/history → full conversation history across all past sessions  
DELETE /chat/{user\_id}/memory → wipe a user's memory (GDPR-style reset)  
GET /catalog → returns the product/pricing catalog the agent uses  
GET /health → service health check

## Three Core Requirements

### 1. Persistent Cross-Session Memory

The agent must remember facts across separate API calls and sessions. If a user asks "what's your enterprise pricing?" in session 1, and then in session 2 asks "does that include SSO?", the agent must know what pricing was already discussed — without it being re-sent in the request body.

Memory must be stored in a DB (SQLite is fine), not in the request payload. Two separate curl calls with the same `user_id` must demonstrate continuity.

### 2. Tool Use — Not Hallucination

The agent must use at least two tools instead of answering from raw LLM knowledge:

| Tool                                                        | Purpose                                                             |
|-------------------------------------------------------------|---------------------------------------------------------------------|
| <code>search_catalog(query)</code>                          | Semantic or keyword search over the product catalog JSON            |
| <code>get_user_memory(user_id)</code>                       | Retrieves relevant past facts about this user for context injection |
| <i>(Bonus)</i> <code>flag_for_human(user_id, reason)</code> | Escalates a conversation if confidence drops below threshold        |

Tools must be defined as real callable functions — not string-injected into the system prompt.

### 3. Self-Evaluation on Every Response

Every `/chat` response must include an `eval` block:

```
{
  "response": "Our Enterprise plan is $499/month and includes SSO and audit logs.",
  "eval": {
    "groundedness": 0.91,
    "relevance": 0.88,
    "confidence": 0.85,
    "flagged": false,
    "reasoning": "Response sourced directly from catalog. User context applied. No hallucination risk detected."
  },
  "tools_called": ["search_catalog", "get_user_memory"],
  "session_id": "uuid"
}
```

The scores don't have to be from a separate eval model — a prompted LLM self-score is fine. What matters is that it's structured, always present, and logged.

## What to Include in the Repo

**Mock product catalog** — they can build it as a `catalog.json`:

```
{
  "plans": [
    { "name": "Starter", "price": "$49/mo", "features": ["5 users", "API access", "email support"] },
    { "name": "Growth", "price": "$199/mo", "features": ["25 users", "webhooks", "priority support"] },
  ]
}
```

```
{ "name": "Enterprise", "price": "$499/mo", "features": ["unlimited users", "SSO", "audit logs", "SLA"] }
]
```

## Architecture Requirements

```
app/
  api/      → route handlers only
  agents/   → agent loop, tool definitions, eval logic
  memory/   → memory read/write layer (abstracted — swappable backend)
  tools/    → search_catalog, get_user_memory, flag_for_human
  services/ → chat service, eval service
  models/   → Pydantic schemas
  db/       → SQLAlchemy models, migrations
main.py
```

The `memory/` layer must be an abstraction — swapping from SQLite to Postgres or Mem0 should require changing one file, not ten.

## Hosting

- Deploy to **Railway**
- Live URL in the README
- Must support two sequential POST calls to `/chat/{user_id}` that demonstrate memory working across calls

## README Requirements

1. **Architecture diagram** — how a user message flows from API → agent → tools → eval → response
2. **Memory design decision** — why they stored memory the way they did, and what they'd use at scale
3. **Eval design** — how they implemented self-scoring, its limitations, what they'd replace it with
4. **Two curl commands** that demonstrate cross-session memory (call 1 sets context, call 2 uses it)
5. **Live Railway URL**

## Evaluation Criteria

| Dimension                | Weight | Signal                                                                                               |
|--------------------------|--------|------------------------------------------------------------------------------------------------------|
| Memory actually persists | 25%    | Two sequential calls with the same user_id prove it. No in-memory dict tricks.                       |
| Tool use is real         | 20%    | <code>search_catalog</code> actually searches. <code>get_user_memory</code> actually queries the DB. |
| Eval block quality       | 20%    | Is it structured? Always present? Logged? Or is it a random number?                                  |
| Code architecture        | 20%    | Memory layer is abstracted. Tools are portable. Clean service separation.                            |
| README depth             | 15%    | Curl demo works. Design decisions documented. Tradeoffs acknowledged.                                |

## Bonus (Optional)

- `GET /chat/{user_id}/evals` — aggregated eval scores across all sessions for a user (what % of responses were high confidence?)
- Memory summarization — instead of storing every message, the agent compresses older context into a summary automatically
- `flagged: true` responses trigger a log entry that a human reviewer endpoint could query

## Submission

- GitHub repo (public)
- Railway live URL
- Two curl commands in the README that work against the live URL
- Demo Video